

Monitoring avec Prometheus & Grafana

Métriques, PromQL, dashboards & alerting — superviser ses applications

Formation DevOps • avec illustrations

Formateur : Haithem Mihoubi

7 modules • fil rouge : superviser nginx • 2026

Table des matières

00	Introduction : l'observabilité	3
01	L'architecture de Prometheus	6
02	Les métriques : types & instrumentation	10
03	PromQL : interroger les métriques	14
04	Grafana : visualiser les métriques	18
05	L'alerting : être prévenu à temps	21
06	Pratique sur Kubernetes & checklist	25

Introduction : l'observabilité

1. Le problème : « est-ce que ça marche ? »

Votre application nginx tourne en production. Mais :

- Combien de requêtes traite-t-elle **par seconde** ?
- Le **taux d'erreurs** augmente-t-il ?
- La **latence** se dégrade-t-elle ?
- La mémoire va-t-elle **saturer** dans une heure ?
- Si ça tombe à 3 h du matin, **qui est prévenu** — et comment ?

Sans **monitoring**, on découvre les pannes... quand les utilisateurs se plaignent. L'objectif est de **voir venir** les problèmes et de **comprendre** ce qui se passe à l'intérieur.

2. Les trois piliers de l'observabilité

Pilier	Question	Exemple
Métriques	« combien ? » (chiffres dans le temps)	1 200 req/s, 2 % d'erreurs, 80 Mo de RAM
Logs	« que s'est-il passé ? » (événements)	« 500 Internal Server Error sur /api »
Traces	« où est passé le temps ? » (parcours d'une requête)	40 ms en base, 10 ms en cache

Ce cours se concentre sur les **métriques** — le pilier le plus mature pour le monitoring et l'alerting — avec le duo standard **Prometheus + Grafana**.

3. La solution : Prometheus + Grafana

Outil	Rôle	Analogie
Prometheus	collecte et stocke les métriques, les interroge, déclenche les alertes	le capteur + la mémoire
Grafana	visualise les métriques en tableaux de bord	l'écran de contrôle
Alertmanager	route et envoie les alertes (mail, Slack...)	la sonnette

Ce trio est devenu le **standard de fait** du monitoring cloud-native, au cœur de l'écosystème Kubernetes (projets de la CNCF, comme Kubernetes lui-même).

4. La chaîne complète

La chaîne complète du monitoring



On instrumente l'application, Prometheus collecte et stocke, Grafana affiche, Alertmanager prévient.

Les 3 piliers de l'observabilité : métriques (ici), logs, traces.

Exposer → collecter → stocker → visualiser → alerter : les cinq maillons du monitoring.

1. **Exposer** : l'application (nginx + un exporter) publie ses métriques sur `/metrics`.
2. **Collecter** : Prometheus va les **chercher** (pull) à intervalle régulier.
3. **Stocker** : il les range dans sa base de séries temporelles (TSDB).
4. **Visualiser** : Grafana interroge Prometheus et trace des courbes.
5. **Alerter** : si une condition est dépassée, Alertmanager **notifie**.

5. Notre fil rouge : superviser nginx

Comme dans les cours précédents, **nginx** sert d'exemple concret. On va :

- exposer les métriques de nginx (via le **nginx-prometheus-exporter**) ;
- les collecter avec **Prometheus** ;
- écrire des requêtes **PromQL** (taux de requêtes, % d'erreurs, latence) ;
- bâtir un **dashboard Grafana** ;
- créer une **alerte** « taux d'erreurs trop élevé » et la router vers Slack.

6. Pourquoi le modèle « pull » de Prometheus ?

Contrairement à d'autres systèmes où l'application **envoie** ses métriques (push), Prometheus **va les chercher** (pull) sur `/metrics`.

Avantage du pull	Conséquence
Prometheus sait si une cible est down	la non-réponse est elle-même un signal
Pas de configuration côté application	l'app expose, Prometheus découvre
Maîtrise du rythme de collecte	un seul endroit règle la fréquence
Simple à tester	il suffit d'ouvrir <code>/metrics</code> dans un navigateur

7. Démarrer en local

```
# Prometheus en conteneur
docker run -p 9090:9090 prom/prometheus
# → interface : http://localhost:9090

# Grafana en conteneur
docker run -p 3000:3000 grafana/grafana
# → interface : http://localhost:3000 (admin / admin)
```

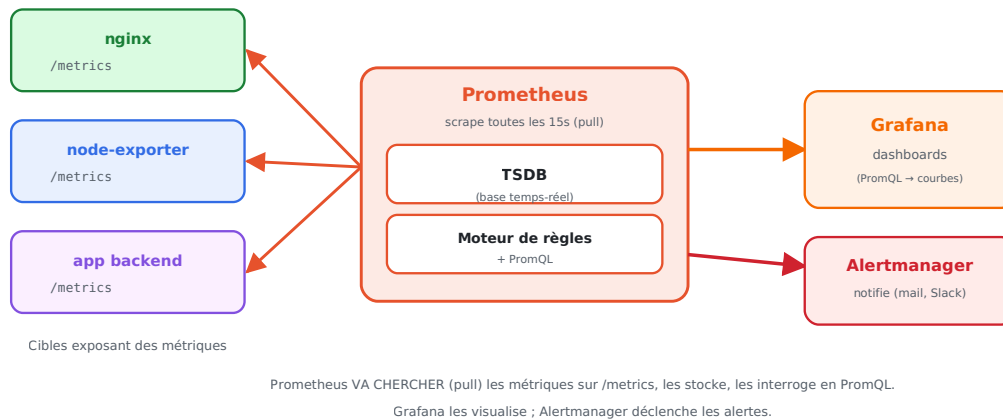
Pré-requis : des bases de Docker et, idéalement, de Kubernetes (le module final déploie la stack sur un cluster). Aucune connaissance préalable de Prometheus n'est requise.

Dans le module suivant, on décortique l'architecture de Prometheus.

L'architecture de Prometheus

Prometheus repose sur une idée simple : **aller chercher** (pull) des métriques exposées en HTTP, les **stocker** dans une base temporelle, et les rendre **interrogeables**.

L'architecture de Prometheus (modèle pull)



Prometheus scrape les cibles sur /metrics, stocke dans la TSDB, et sert Grafana & Alertmanager.

1. Le scraping : le cœur du système

Prometheus interroge périodiquement (par défaut **toutes les 15 s**) l'endpoint `/metrics` de chaque **cible** (target). Cet endpoint renvoie un texte simple :

```
# HELP nginx_http_requests_total Nombre total de requêtes HTTP
# TYPE nginx_http_requests_total counter
nginx_http_requests_total{method="GET",status="200"} 1027
nginx_connections_active 12
```

Chaque ligne = une **métrique** + ses **labels** + sa **valeur**. Prometheus lit, horodate et stocke. C'est tout.

2. Les composants

Composant	Rôle
Serveur Prometheus	scrape, stocke (TSDB), évalue les règles, expose PromQL
TSDB	base de séries temporelles locale (sur disque)
Exporters	traduisent l'état d'un système en métriques <code>/metrics</code>
Pushgateway	pour les jobs courts qui ne peuvent pas être « pullés »
Alertmanager	reçoit les alertes, les route et notifie (module 05)

3. La configuration : `prometheus.yml`

Tout se déclare dans un fichier YAML. Le concept clé : les **jobs** de scraping.

```
global:
  scrape_interval: 15s          # fréquence de collecte par défaut
  evaluation_interval: 15s      # fréquence d'évaluation des règles

scrape_configs:
  - job_name: 'nginx'
    static_configs:
      - targets: ['nginx-exporter:9113']    # où scraper

  - job_name: 'prometheus'
    static_configs:
      - targets: ['localhost:9090']         # Prometheus se surveille lui-même
```

Recharger la configuration sans redémarrer :

```
curl -X POST http://localhost:9090/-/reload    # si --web.enable-lifecycle
```

4. Vérifier les cibles

Dans l'interface web (`http://localhost:9090`), le menu **Status ► Targets** liste chaque cible et son état :

État	Signification
UP	la cible répond, métriques collectées
DOWN	la cible ne répond pas → c'est déjà une information

Le « **down** » est un **signal**. Comme Prometheus va chercher les métriques, l'absence de réponse indique en soi un problème (cible tombée, réseau coupé). On peut alerter dessus avec la métrique spéciale `up == 0`.

5. La découverte dynamique (service discovery)

Lister les cibles « en dur » (`static_configs`) ne tient pas sur un cluster où les Pods vont et viennent. Prometheus sait **découvrir** automatiquement les cibles :

Mécanisme	Découvre...
<code>kubernetes_sd_configs</code>	les Pods/Services/Nodes d'un cluster K8s
<code>dns_sd_configs</code>	via des enregistrements DNS
<code>consul_sd_configs</code>	via Consul
<code>file_sd_configs</code>	via des fichiers générés

```
scrape_configs:
- job_name: 'kubernetes-pods'
  kubernetes_sd_configs:
    - role: pod          # découvre tous les Pods automatiquement
```

→ tout Pod **annoté** `prometheus.io/scrape: "true"` est collecté **automatiquement**, sans toucher à la config. C'est ce qui rend Prometheus naturel sur Kubernetes.

6. Le stockage TSDB

- Les données sont stockées **localement**, optimisées pour les séries temporelles.
- Rétention par défaut : **15 jours** (réglable via `--storage.tsdb.retention.time`).
- Pour du **long terme** ou du **multi-cluster**, on ajoute un stockage distant (**Thanos**, **Mimir**, **Cortex**).

7. L'interface intégrée

`http://localhost:9090` offre déjà :

- un **explorateur de requêtes** PromQL (onglet *Graph*) ;
- l'état des **cibles** (*Status ▶ Targets*) ;
- les **règles** et **alertes** actives (*Alerts*) ;
- la configuration chargée (*Status ▶ Configuration*).

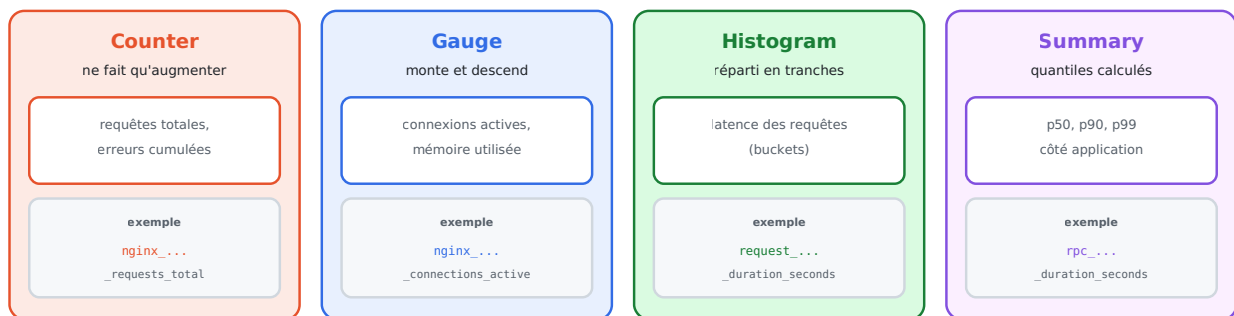
Cette UI suffit pour explorer et déboguer. Pour des **tableaux de bord** présentables et partagés, on branche **Grafana** (module 04).

À retenir : Prometheus **pull** sur `/metrics`, stocke en TSDB, et découvre ses cibles dynamiquement sur Kubernetes. Le module suivant : **ce que** contient `/metrics`.

Les métriques : types & instrumentation

Une métrique Prometheus est un **nombre mesuré dans le temps**, identifié par un **nom** et des **labels**. Tout repose sur quatre types fondamentaux.

Les quatre types de métriques Prometheus



Counter et Gauge couvrent 90 % des besoins. Histogram pour les latences.

Counter, Gauge, Histogram, Summary : choisir le bon type selon ce qu'on mesure.

1. Anatomie d'une métrique

```
nginx_http_requests_total{method="GET", status="200"} 1027
```

Diagramme d'analyse de la métrique ci-dessus :

- `nginx_http_requests_total` : nom de la métrique
- `{method="GET", status="200"}` : labels (dimensions)
- `1027` : valeur

- **Nom** : ce qu'on mesure (convention : `unité` en suffixe, ex. `_total`, `_seconds`, `_bytes`).
- **Labels** : des **dimensions** qui découpent la métrique (par méthode, par statut, par instance...). Ils permettent de filtrer et d'agréger.
- **Valeur** : le nombre, horodaté au moment du scrape.

Attention à la cardinalité. Chaque combinaison de labels crée une **série** distincte. Mettre un label à forte variabilité (un ID utilisateur, une URL unique) fait **exploser** le nombre de séries et sature Prometheus. Labels = **petites** catégories stables.

2. Les quatre types de métriques

Counter — ne fait qu'augmenter

Un compteur **cumulatif** qui ne peut que **croître** (ou repartir de zéro au redémarrage).

```
nginx_http_requests_total    # requêtes totales depuis le démarrage
nginx_http_errors_total     # erreurs cumulées
```

On ne lit presque jamais sa valeur brute : on calcule son **taux** avec `rate()` (module 03).

Gauge — monte et descend

Une valeur **instantanée** qui peut augmenter **ou** diminuer.

```
nginx_connections_active    # connexions ouvertes maintenant
node_memory_used_bytes     # mémoire utilisée
node_cpu_usage_ratio        # charge CPU
```

Histogram — réparti en tranches (buckets)

Mesure une **distribution** (typiquement des **latences**) en comptant les observations par **tranches** (`_bucket`), plus une somme (`_sum`) et un nombre (`_count`).

```
http_request_duration_seconds_bucket{le="0.1"}  980    # ≤ 100 ms
http_request_duration_seconds_bucket{le="0.5"}  1015   # ≤ 500 ms
http_request_duration_seconds_sum                52.3
http_request_duration_seconds_count              1027
```

→ permet de calculer un **quantile** (p95, p99) avec `histogram_quantile()`.

Summary — quantiles pré-calculés

Comme l'histogram, mais les **quantiles** sont calculés **côté application**. Plus précis mais non agrégeable entre instances. On lui préfère souvent l'**histogram**.

Lequel choisir ?

Je mesure...	Type
un total qui s'accumule (requêtes, erreurs, octets)	Counter
une valeur qui varie (connexions, mémoire, température)	Gauge
une distribution / des latences agrégeables	Histogram
des quantiles précis sur une seule instance	Summary

3. Exposer des métriques : les exporters

Beaucoup de logiciels n'exposent pas nativement du format Prometheus. Un **exporter** fait la traduction : il lit l'état du système et publie un `/metrics`.

Exporter	Expose les métriques de...
node-exporter	la machine (CPU, RAM, disque, réseau)
nginx-prometheus-exporter	nginx (notre cas)
blackbox-exporter	sondes externes (ping, HTTP, TLS)
postgres / redis exporter	bases de données

Notre cas : superviser nginx

nginx publie un endpoint d'état (`stub_status`) ; l'exporter le convertit en métriques Prometheus.

```
# nginx avec le module stub_status activé, puis l'exporter en sidecar
docker run -p 9113:9113 nginx/nginx-prometheus-exporter:latest \
  -nginx.scrape-uri=http://nginx:8080/stub_status
# → métriques disponibles sur http://localhost:9113/metrics
```

```
nginx_connections_active 12
nginx_connections_accepted_total 9876
nginx_http_requests_total 10234
```

4. Instrumenter sa propre application

Pour une application maison, on utilise une **bibliothèque client** Prometheus (Go, Python, Java, Node...). Exemple en Python :

```
from prometheus_client import Counter, start_http_server

REQUESTS = Counter('app_requests_total', 'Requêtes', ['method', 'status'])

def handle(req):
    REQUESTS.labels(method=req.method, status='200').inc()

start_http_server(8000)  # expose /metrics sur le port 8000
```

5. La méthode RED : quoi mesurer en priorité

Pour un service comme nginx, trois métriques suffisent à savoir s'il va bien :

Lettre	Métrique	Avec nginx
Rate	requêtes par seconde	<code>rate/nginx_http_requests_total[5m])</code>
Errors	taux d'erreurs	part des <code>status="5.."</code>
Duration	latence	histogram des durées de requête

(Pour les **ressources** d'une machine, la méthode sœur **USE** — Utilization, Saturation, Errors — guide ce qu'il faut surveiller : CPU, mémoire, disque.)

À retenir : nom + labels + valeur ; quatre types (Counter/Gauge/Histogram/Summary) ; on expose via des **exporters** ; on priorise avec **RED**. Place aux requêtes : **PromQL**.

PromQL : interroger les métriques

PromQL (Prometheus Query Language) est le langage qui transforme des séries brutes en informations utiles : taux, pourcentages, agrégations, quantiles.

Anatomie d'une métrique & requêtes PromQL

`nginx_http_requests_total{method="GET", status="200"} 42`

nom de la métrique labels (dimensions) valeur

Taux de requêtes / s (sur 5 min)

```
rate(nginx_http_requests_total[5m])
```

Filtrer par label

```
nginx_http_requests_total{status="500"}
```

Total par méthode

```
sum by (method) (rate(nginx_http_requests_total[5m]))
```

% d'erreurs 5xx

```
sum(rate(...{status=~"5.."}[5m])) / sum(rate(...[5m]))
```

Latence p95

```
histogram_quantile(0.95, rate(req_duration_seconds_bucket[5m]))
```

D'une métrique brute aux requêtes qui répondent aux vraies questions.

1. La requête la plus simple : sélectionner

```
nginx_http_requests_total
```

→ renvoie **toutes** les séries de cette métrique (une par combinaison de labels).

Filtrer par label

```
nginx_http_requests_total{status="500"}           # égalité
nginx_http_requests_total{status!="200"}           # différent
nginx_http_requests_total{status=~"5.."}           # regex : tout 5xx
nginx_http_requests_total{method="GET", status="200"} # plusieurs labels
```

2. Les quatre types de résultats

Type	Quoi	Exemple
Instant vector	une valeur par série, à l'instant t	<code>nginx_connections_active</code>
Range vector	une plage de valeurs sur une durée	<code>nginx_http_requests_total[5m]</code>
Scalar	un nombre simple	<code>42</code>
String	une chaîne	(rare)

3. `rate()` : le taux par seconde d'un counter

La fonction la plus utilisée. Un counter ne fait qu'augmenter : sa valeur brute n'a pas d'intérêt. Ce qu'on veut, c'est sa **vitesse de croissance**.

```
rate(nginx_http_requests_total[5m])
```

→ « combien de requêtes **par seconde**, en moyenne sur les 5 dernières minutes ». C'est la métrique **R** de RED (le débit).

`rate` exige un **range vector** (`[5m]`) et un **counter**. Il gère automatiquement les redémarrages (remise à zéro du compteur). Pour les pics brefs, `irate()` est plus réactif.

4. Agréger avec `sum`, `avg`, `by`, `without`

Les **opérateurs d'agrégation** combinent plusieurs séries.

```
# Total des requêtes/s, toutes séries confondues
sum(rate(nginx_http_requests_total[5m]))

# Requetes/s regroupées PAR méthode
sum by (method) (rate(nginx_http_requests_total[5m]))

# Moyenne d'utilisation CPU par instance
avg by (instance) (rate(node_cpu_seconds_total[5m]))
```

Opérateur	Effet
<code>sum</code> , <code>avg</code> , <code>min</code> , <code>max</code>	agrègent les valeurs
<code>count</code>	compte les séries
<code>by (label)</code>	groupe par ce(s) label(s)
<code>without (label)</code>	agrège en ignorant ce label
<code>topk(3, ...)</code>	le top 3

5. Calculer un pourcentage d'erreurs

Un grand classique — la métrique **E** de RED :

```
sum(rate(nginx_http_requests_total{status=~"5.."}[5m]))
/
sum(rate(nginx_http_requests_total[5m]))
```

→ proportion de requêtes en erreur 5xx. Multipliée par 100, c'est le % **d'erreurs**, parfait pour une alerte (« > 5 % pendant 10 min »).

6. Les quantiles de latence (histogram)

La métrique **D** de RED, à partir d'un histogram :

```

histogram_quantile(
  0.95,
  sum by (le) (rate(http_request_duration_seconds_bucket[5m]))
)

```

→ la **latence p95** : 95 % des requêtes sont plus rapides que cette valeur.

7. Opérations et comparaisons

```

# Arithmétique
node_memory_used_bytes / node_memory_total_bytes * 100      # % mémoire

# Comparaisons (utiles pour les alertes – renvoie les séries qui matchent)
node_filesystem_avail_bytes < 10e9                          # disque < 10 Go
up == 0                                                        # cibles DOWN

```

8. Tableau récapitulatif des fonctions clés

Fonction	Usage
<code>rate(counter[5m])</code>	taux/s moyen d'un counter
<code>irate(counter[5m])</code>	taux/s instantané (pics)
<code>increase(counter[1h])</code>	augmentation absolue sur la période
<code>sum / avg / max by (l)</code>	agrégation groupée
<code>histogram_quantile(0.95, ...)</code>	quantile depuis un histogram
<code>topk(5, ...)</code>	les 5 plus grandes valeurs
<code>predict_linear(metric[1h], 4*3600)</code>	prévoir la valeur dans 4 h

9. Tester ses requêtes

Dans l'UI Prometheus (<http://localhost:9090> , onglet **Graph**) : on tape la requête, on voit la **table** et la **courbe** immédiatement. C'est le terrain de jeu idéal pour construire ses requêtes avant de les mettre dans un dashboard ou une alerte.

À retenir : `rate()` sur les counters, agrégation avec `sum ... by`, % d'erreurs par division, `histogram_quantile` pour les latences. Ces quatre patterns couvrent l'essentiel. Place à la **visualisation** : Grafana.

Grafana : visualiser les métriques

Prometheus stocke et calcule ; **Grafana** transforme ces chiffres en **tableaux de bord** lisibles, partageables et dynamiques.

1. Le rôle de Grafana

- Se connecte à une **source de données** (Prometheus, mais aussi Loki, Elasticsearch...).
- Affiche des **panneaux** (panels) : courbes, jauges, compteurs, tableaux.
- Organise les panneaux en **dashboards** réutilisables.
- Offre des **variables**, des **annotations** et ses propres **alertes**.

Grafana ne stocke pas les métriques : il **interroge** Prometheus en PromQL à chaque affichage. C'est une **fenêtre** sur les données, pas un entrepôt.

2. Connecter Prometheus comme source de données

Dans Grafana (<http://localhost:3000> , défaut `admin / admin`) :

Configuration ▶ **Data sources** ▶ **Add** ▶ **Prometheus**, puis l'URL :

```
http://prometheus:9090      # nom du service Prometheus (ou localhost:9090)
```

On peut aussi la déclarer en **YAML** (provisioning, versionnable) :

```
# datasources.yaml
apiVersion: 1
datasources:
- name: Prometheus
  type: prometheus
  access: proxy
  url: http://prometheus:9090
  isDefault: true
```

3. Créer un dashboard pour nginx

Dashboards ▶ **New** ▶ **Add visualization**, choisir la source Prometheus, puis saisir une requête PromQL par panneau.

Panneau	Requête PromQL	Type de visu
Requêtes/s	<code>sum(rate/nginx_http_requests_total[5m]))</code>	Time series
% d'erreurs 5xx	<code>sum(rate(...{status=~"5.."}[5m])) / sum(rate(...[5m]))</code>	Gauge / Stat
Connexions actives	<code>nginx_connections_active</code>	Time series
Latence p95	<code>histogram_quantile(0.95, ...)</code>	Time series

4. Les types de panneaux courants

Type	Pour
Time series	l'évolution dans le temps (le plus utilisé)
Stat	une grande valeur unique (req/s actuelles)
Gauge	une valeur sur une échelle (% CPU, % erreurs)
Bar gauge	comparer des catégories
Table	données détaillées par série
Heatmap	distribution (idéal pour les histogramms de latence)

5. Les variables : des dashboards dynamiques

Plutôt qu'un dashboard figé, on ajoute des **variables** (menus déroulants) pour filtrer.

```
# Variable $instance : liste des instances disponibles
label_values/nginx_http_requests_total, instance)

# Dans les panneaux, on filtre par la variable
rate/nginx_http_requests_total{instance="$instance"}[5m])
```

→ un **seul** dashboard sert pour **toutes** les instances : on choisit dans le menu.

6. Ne pas réinventer la roue : importer des dashboards

Des milliers de dashboards prêts à l'emploi existent sur grafana.com/dashboards. On les importe par leur **ID**.

Dashboards ► Import, entrer l'ID (ex. *NGINX exporter* ou *Node Exporter Full*), choisir la source Prometheus. Le dashboard apparaît, complet.

Dashboard	ID indicatif
Node Exporter Full	1860
NGINX (exporter)	12708
Kubernetes cluster	7249

7. Bonnes pratiques de dashboard

- **Une intention par dashboard** : « santé de nginx », « ressources du cluster »...
- Placer les **métriques RED** en haut (débit, erreurs, latence).
- Des **unités** correctes (req/s, %, ms, octets) pour des axes lisibles.
- Des **seuils** colorés (vert/orange/rouge) pour repérer l'anormal d'un coup d'œil.
- **Versionner** les dashboards en JSON (provisioning) plutôt que les cliquer à la main.

```
# dashboards provisioning : Grafana charge les JSON d'un dossier
apiVersion: 1
providers:
- name: 'default'
  folder: 'nginx'
  type: file
  options:
    path: /var/lib/grafana/dashboards
```

8. Grafana vs l'UI Prometheus

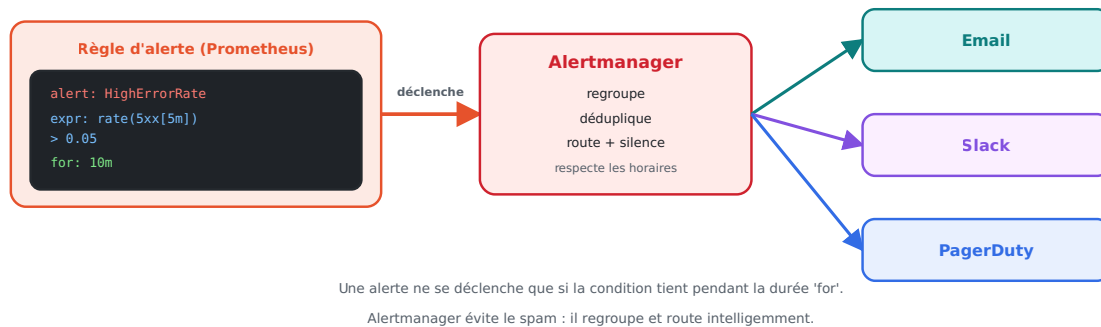
	UI Prometheus	Grafana
Explorer/déboguer une requête	✓ rapide	✓ (Explore)
Dashboards présentables	✗	✓
Partage, variables, seuils	✗	✓
Multi-sources (logs, métriques)	✗	✓

À retenir : Grafana = la couche de visualisation. On branche Prometheus en source, on compose des panneaux PromQL (priorité aux métriques RED), on rend tout dynamique avec des variables, et on importe des dashboards existants pour gagner du temps.

L'alerting : être prévenu à temps

Visualiser ne suffit pas : personne ne regarde un dashboard à 3 h du matin. L'**alerting** déclenche une notification **automatique** quand une condition est dépassée.

Le pipeline d'alerting



Prometheus évalue les règles ; Alertmanager regroupe, route et notifie.

1. Deux étapes, deux composants

Étape	Composant	Rôle
Détecter	Prometheus	évalue des règles PromQL en continu
Notifier	Alertmanager	regroupe, route et envoie (mail, Slack...)

Cette séparation est volontaire : Prometheus dit **quoi** se passe ; Alertmanager décide **qui** prévenir et **comment**.

2. Écrire une règle d'alerte

Les règles vivent dans un fichier chargé par Prometheus (`rule_files:` dans `prometheus.yml`).

```
groups:
- name: nginx
  rules:
  - alert: HighErrorRate
    expr: |
      sum(rate/nginx_http_requests_total{status=~"5.."}[5m]))
      / sum(rate/nginx_http_requests_total[5m])) > 0.05
    for: 10m # la condition doit tenir 10 min
    labels:
      severity: critical
    annotations:
      summary: "Taux d'erreurs 5xx élevé sur nginx"
      description: "Plus de 5 % d'erreurs depuis 10 minutes."
```

Les champs clés

Champ	Rôle
<code>alert</code>	nom de l'alerte
<code>expr</code>	la condition PromQL ; l'alerte se déclenche si elle renvoie des séries
<code>for</code>	durée pendant laquelle la condition doit persister avant de déclencher
<code>labels</code>	métadonnées (ex. <code>severity</code>) servant au routage
<code>annotations</code>	texte humain (<code>summary</code> , <code>description</code>) pour la notification

Le champ `for` évite les faux positifs. Un pic d'erreurs d'une seconde ne réveille personne ; seul un problème qui **dure** 10 minutes déclenche l'alerte.

3. Le cycle de vie d'une alerte

État	Signification
<code>inactive</code>	la condition est fausse
<code>pending</code>	la condition est vraie, mais <code>for</code> n'est pas encore écoulé
<code>firing</code>	la condition tient depuis <code>for</code> → envoyée à Alertmanager

On suit ces états dans l'UI Prometheus, onglet **Alerts**.

4. Alertmanager : router intelligemment

Une fois l'alerte `firing`, Alertmanager prend le relais. Sa configuration (`alertmanager.yml`) définit le **routage** et les **destinataires**.

```

route:
  receiver: 'equipe-ops'           # destinataire par défaut
  group_by: ['alertname']          # regrouper les alertes similaires
  group_wait: 30s                  # attendre 30s avant le premier envoi
  repeat_interval: 4h              # ré-notifier toutes les 4h si non résolu
  routes:
    - match:
        severity: critical          # les alertes critiques...
        receiver: 'pagerduty'       # ...vont vers l'astreinte

receivers:
  - name: 'equipe-ops'
    slack_configs:
      - channel: '#alertes'
        api_url: '<webhook-slack>'
  - name: 'pagerduty'
    pagerduty_configs:
      - service_key: '<clé>'

```

5. Ce qu'Alertmanager apporte (et qu'une simple notification n'a pas)

Fonction	Pourquoi c'est essentiel
Regroupement (<code>group_by</code>)	50 Pods qui tombent = 1 notification, pas 50
Déduplication	la même alerte de plusieurs Prometheus = une seule notif
Routage (<code>routes</code>)	critique → astreinte, info → Slack
Silences	couper le bruit pendant une maintenance planifiée
Inhibition	si « cluster down », taire les alertes « Pod down » dérivées
<code>repeat_interval</code>	rappeler tant que ce n'est pas résolu

6. Bonnes pratiques d'alerting

- **Alerter sur les symptômes, pas les causes** : « le service répond en erreur » (vécu par l'utilisateur) plutôt que « le CPU est à 90 % » (qui n'est pas forcément un problème).
- **Toujours un `for`** pour éviter les alertes qui clignotent.
- **Des annotations utiles** : que faire ? quel dashboard consulter ? (runbook).
- **Graduer la `severity`** : `critical` réveille quelqu'un, `warning` attend le matin.
- **Tester les alertes** : provoquer la condition pour vérifier que la notif arrive.
- **Éviter la fatigue d'alerte** : trop d'alertes = on les ignore. Mieux vaut **peu** d'alertes pertinentes.

7. Une alerte universelle : la cible est tombée

```
- alert: TargetDown
  expr: up == 0
  for: 5m
  labels: { severity: critical }
  annotations:
    summary: "Cible {{ $labels.job }} injoignable"
```

Grâce au modèle pull, `up == 0` détecte qu'une cible (nginx, un node...) ne répond plus — souvent la **première** chose à surveiller.

À retenir : Prometheus **détecte** via des règles PromQL avec un `for` ; Alertmanager **route et notifie** sans spammer. On alerte sur les **symptômes** visibles par l'utilisateur, avec parcimonie. C'est le maillon qui rend le monitoring **actionnable**.

Pratique sur Kubernetes & checklist

On rassemble tout : déployer la stack de monitoring sur un cluster Kubernetes, superviser nginx de bout en bout, puis une checklist finale.

1. La voie rapide : kube-prometheus-stack

Sur Kubernetes, on **n'installe pas** Prometheus, Grafana et Alertmanager à la main. On déploie le **chart Helm** `kube-prometheus-stack`, qui livre l'ensemble **préconfiguré et intégré**.

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update

helm install monitoring prometheus-community/kube-prometheus-stack \
  --namespace monitoring --create-namespace
```

On obtient d'un coup :

Composant	Rôle
Prometheus	collecte (déployé via l' Operator)
Grafana	dashboards (avec des dashboards K8s pré-installés)
Alertmanager	alerting
node-exporter	métriques de chaque node
kube-state-metrics	état des objets K8s (Pods, Deployments...)

```
# Accéder à Grafana
kubectl port-forward -n monitoring svc/monitoring-grafana 3000:80
# → http://localhost:3000 (admin / prom-operator)
```

2. Le Prometheus Operator : surveiller de façon déclarative

L'Operator introduit des objets Kubernetes dédiés. Au lieu d'éditer `prometheus.yml`, on **déclare** quoi surveiller avec un **ServiceMonitor**.

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: nginx
  namespace: monitoring
  labels:
    release: monitoring      # pour que Prometheus l'adopte
spec:
  selector:
    matchLabels:
      app: nginx              # cible le Service nginx
  endpoints:
    - port: metrics          # le port nommé "metrics" du Service
      interval: 15s
```

C'est du monitoring déclaratif, cohérent avec Kubernetes : on crée un Service nginx + un ServiceMonitor, et Prometheus se met à le scraper **automatiquement**. Pas de rechargement de config manuel.

3. Superviser nginx de bout en bout

```
# 1) Déployer nginx avec son exporter (sidecar) – Deployment + Service
kubectl apply -f nginx-with-exporter.yaml -n monitoring

# 2) Déclarer le ServiceMonitor
kubectl apply -f nginx-servicemonitor.yaml -n monitoring

# 3) Vérifier que Prometheus voit la cible
#   UI Prometheus ▶ Status ▶ Targets → nginx = UP

# 4) Dans Grafana : importer le dashboard "NGINX exporter" (ID 12708)

# 5) Créer une PrometheusRule pour l'alerte taux d'erreurs
kubectl apply -f nginx-alert-rule.yaml -n monitoring
```

Règle d'alerte au format Operator ([PrometheusRule](#)) :

```

apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: nginx-rules
  labels: { release: monitoring }
spec:
  groups:
    - name: nginx
      rules:
        - alert: NginxHighErrorRate
          expr: |
            sum(rate/nginx_http_requests_total{status=~"5.."}[5m]))
            / sum(rate/nginx_http_requests_total[5m])) > 0.05
          for: 10m
          labels: { severity: critical }
          annotations:
            summary: "nginx : taux d'erreurs 5xx > 5 %"

```

4. Que surveiller en priorité (rappel)

Niveau	Méthode	Métriques
Service (nginx)	RED	Rate (req/s), Errors (% 5xx), Duration (latence p95)
Machine / node	USE	Utilization, Saturation, Errors (CPU, RAM, disque)
Kubernetes	—	Pods non prêts, redémarrages, <code>up == 0</code>

5. Erreurs fréquentes & solutions

Symptôme	Cause probable	Solution
Cible <code>DOWN</code>	endpoint <code>/metrics</code> injoignable	vérifier le port, le réseau, l'exporter
Aucune métrique nginx	exporter mal configuré	tester <code>curl <exporter>:9113/metrics</code>
ServiceMonitor ignoré	label <code>release</code> manquant	aligner sur le label attendu par Prometheus
Trop de séries, Prometheus lent	cardinalité des labels	retirer les labels à forte variabilité
Grafana « No data »	mauvaise source / requête	vérifier la data source et le PromQL dans <i>Explore</i>
Alerte jamais déclenchée	<code>for</code> trop long / <code>expr</code> fausse	tester l' <code>expr</code> dans l'UI Prometheus
Pluie d'alertes	pas de <code>for</code> / pas de regroupement	ajouter <code>for</code> , <code>group_by</code> dans Alertmanager

6. Checklist d'un monitoring sain

- [] Stack déployée (`kube-prometheus-stack` ou équivalent).
 - [] Chaque service expose `/metrics` (natif ou via un **exporter**).
 - [] Cibles **UP** dans *Status* ► *Targets*.
 - [] Métriques **RED** suivies pour chaque service clé (nginx).
 - [] Métriques **USE** suivies pour les nodes.
 - [] Dashboards Grafana en place (importés ou versionnés en JSON).
 - [] Alertes sur les **symptômes** (erreurs, latence, `up == 0`), avec un `for`.
 - [] Alertmanager routé vers un vrai canal (Slack/mail/astreinte) et **testé**.
 - [] **Cardinalité** des labels maîtrisée.
 - [] Rétention et stockage long terme pensés si besoin (Thanos/Mimir).
-

Conclusion

Avec **nginx** comme fil rouge, vous avez bâti une chaîne de monitoring complète : exposer des **métriques**, les **collecter** avec Prometheus (modèle pull), les interroger en **PromQL**, les **visualiser** dans Grafana, et **alerter** via Alertmanager — le tout déployé de façon **déclarative** sur Kubernetes.

Le monitoring n'est pas un luxe : c'est ce qui permet de **voir venir** les problèmes, de **comprendre** les pannes et de **dormir tranquille** parce qu'on sera prévenu à temps.

Pour aller plus loin : les **logs** avec Loki (et l'exploration corrélée dans Grafana), les **traces** distribuées avec Tempo/Jaeger (OpenTelemetry), les **SLO/SLI** et budgets d'erreur, et le stockage longue durée multi-cluster (Thanos, Mimir).